Глава 5. Иерархии классов и обобщения языка C#

МГТУ им. Н.Э. Баумана
Факультет Информатика и системы управления
Кафедра Компьютерные системы и сети
Дисциплина Алгоритмизация и программирование
Лектор: д.т.н., проф. Иванова Галина Сергеевна

Отношения между классами

- **Наследование** – класс-потомок наследует компоненты базового класса и может добавлять свои.

- **Реализация** – класс содержит методы, объявленные некоторым интерфейсом или абстрактным базовым классом.

- **Ассоциация** – объект(ы) класса включаются в качестве полей в объекты другого класса. Бывают: двусторонние, односторонние, самоассоциации и многозначные ассоциации. Могут иметь название, например «Владеет».

  - ***Агрегация***– объект(ы) класс-части включается в объект класса-целое и при отсутствии объектов класса-части объект класса-целого не теряет функциональности.

  - ***Композиция*** – объект(ы) класс-части включается в объект класса-целое и при отсутствии объектов класса-части объект класса-целого теряет функциональность.

- **Зависимость** – методы одного класса используют методы другого.

5.1 Наследование

Синтаксис описания классов-потомков:

[Атрибуты][Спецификаторы] `class` Имя [:Предки] Тело

В качестве предка помимо ***единственного*** обычного класса также могут выступать классы-интерфейсы.

Если предки при объявлении наследника не указаны, то предком является класс Object – предок всех классов C#.

Конструкторы не наследуются. При конструировании объектов реализуется следующая последовательность:

- при отсутствии в конструкторе явного вызова конструктора базового класса именем base, автоматически вызывается конструктор базового класса без параметров;
- затем вызываются конструкторы полей – объектов;
- и последним – конструктор производного класса.

При необходимости любой компонент класса при наследовании можно заменить, используя new. Однако доступ к компоненту базового класса при этом сохраняется:

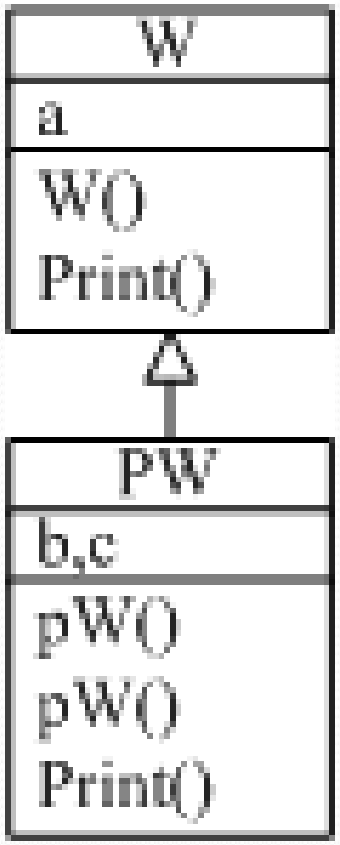
`base.`Имя_компонента

Пример наследования и перегрузки методов

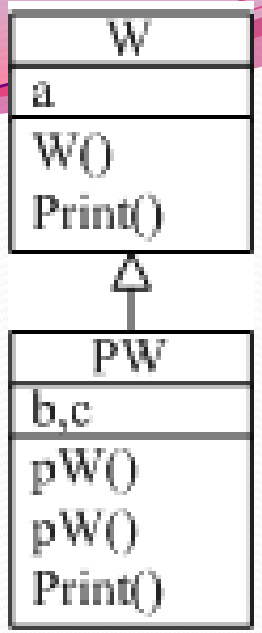
Ex5.01

```
using System;

namespace Ex5_01
{
    class W
    {
        private int a;
        public W(int aa)  // Конструктор
        {
            a = aa;
        }
        public void Print()
        {
            Console.WriteLine("a=" + a);
        }
    }
```



Пример (продолжение)




```
class PW:W
    {
        private int b, c;
        public PW(int aa,int bb):base(aa)
        {
            b = bb; c = aa + b;
        }
        public PW(int aa, int bb, int cc) : this(aa,bb)
        {
            c = cc;
        }
        public new void Print()
        {
            base.Print();
            Console.WriteLine("b=" + b);
            Console.WriteLine("c=" + c);
        }
    }
```

Конструктор базового класса

Перегрузка метода базового класса

Перегруженный конструктор этого же класса

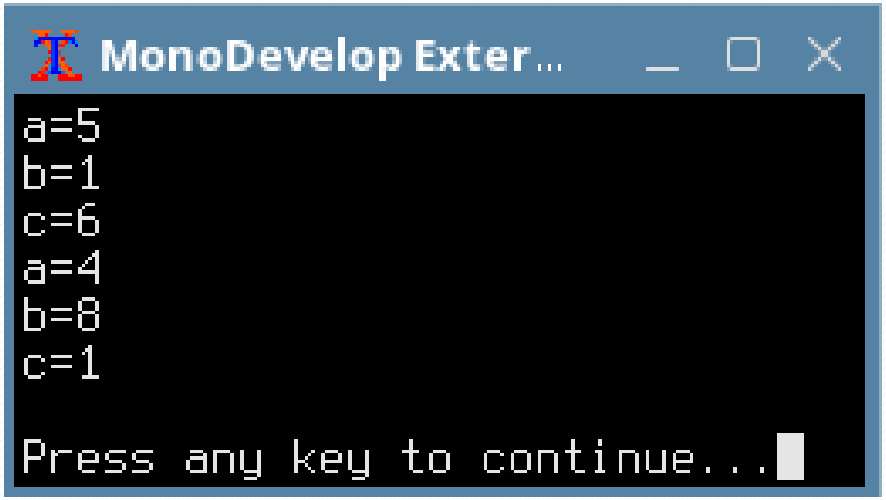
Вызов перегруженного метода базового класса

Пример (окончание)

```
class Program
    {
        static void Main(string[] args)
        {
            PW pw1 = new PW(5,1);
            pw1.Print();
            PW pw2 = new PW(4,8,1);
            pw2.Print();
        }
    }
}
```

Вызов конструктора с двумя параметрами

Вызов конструктора с тремя параметрами



5.2 Полиморфное наследование

Поскольку в C# работа с объектами осуществляется через ссылки, велика вероятность необходимости использования сложного полиморфизма по 2-му случаю.

Виртуальные методы описываются служебным словом **virtual** и вызываются в процессе выполнения программы через ТВМ. Связь объекта с ТВМ реализуется скрытым кодом, вставленным в конструктор.

Для переопределения виртуального метода используются служебное слово **override**.

Так, если в предыдущем примере работать через указатель на объекты базового класса, то получим ошибку:

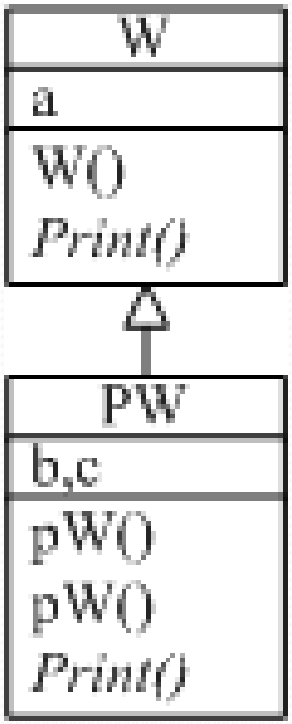
```
class Program
    {
        static void Main(string[] args)
        {
            W w = new PW(4,8,1);  //объект производного класса
                                  //указатель - базового
            w.Print();   // будет вызван метод базового класса!!!
        }
    }
}
```

Пример полиморфного наследования

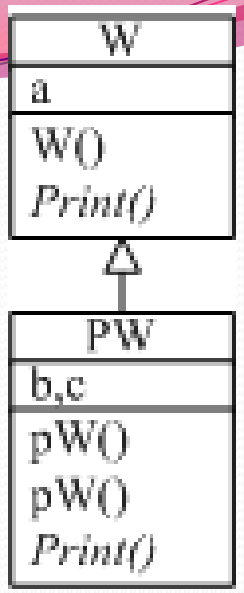
Ex5.02

```
using System;

namespace Ex5_02
{
    class W
    {
        private int a;
        public W(int aa)
        {
            a = aa;
        }
        virtual public void Print()
        {
            Console.WriteLine("a=" + a);
        }
    }
```



Пример (продолжение)




```
class PW:W
    {
        private int b, c;
        public PW(int aa,int bb):base(aa)
        {
            b = bb; c = aa + b;
        }
        public PW(int aa, int bb, int cc) : this(aa,bb)
        {
            c = cc;
        }
        override public new void Print()
        {
            base.Print();
            Console.WriteLine("b=" + b);
            Console.WriteLine("c=" + c);
        }
    }
```

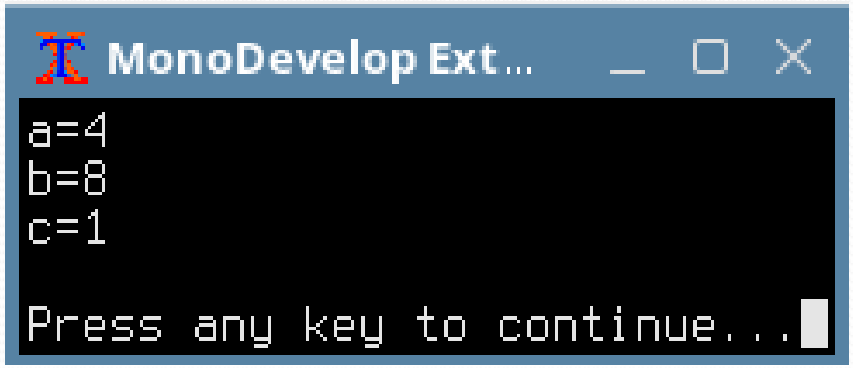
Полиморфная перегрузка метода базового класса

Убираем new, так как это не новый метод

Вызов перегруженного метода базового класса

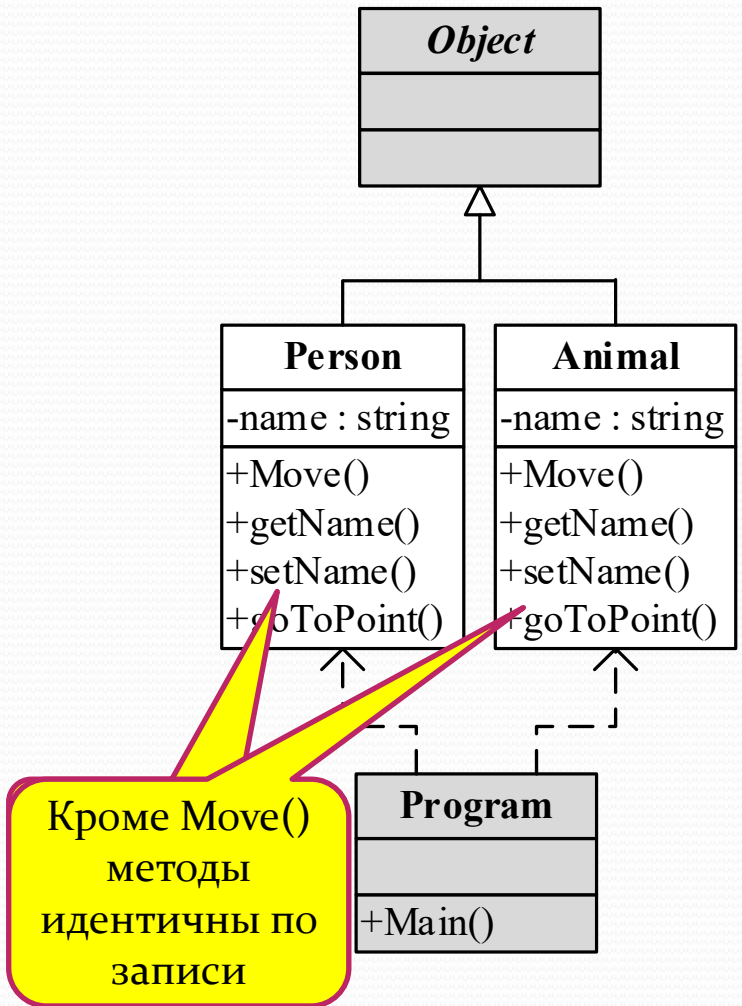
Пример (окончание)

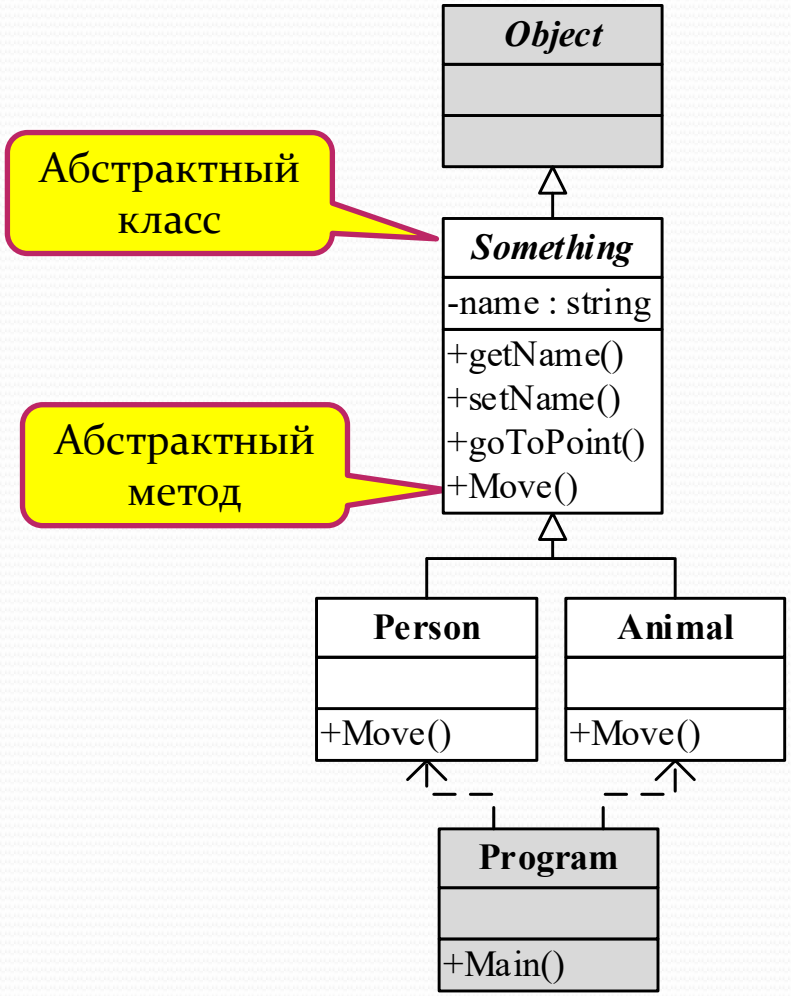
```
class Program
    {
        static void Main(string[] args)
        {
            W w = new PW(4,8,1);
            w.Print();  // будет вызван метод производного класса!!!
        }
    }
}
```



5.3 Абстрактные классы и методы

Методы, не реализованные в классе, называют *абстрактными* abstract. Классы, содержащие абстрактные методы, являются *абстрактными*, объекты таких классов создавать запрещено.





Пример реализации без абстрактного класса

Ex5.03

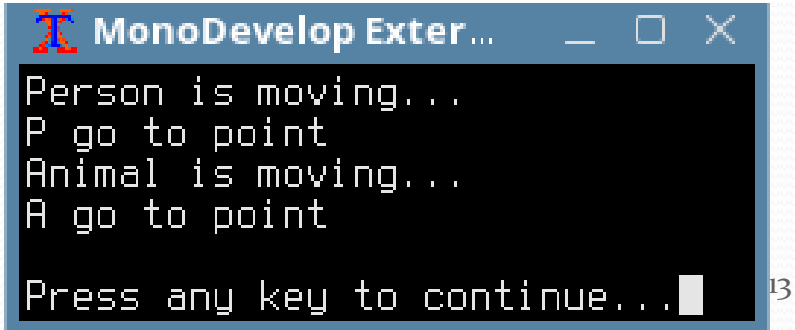
```
using System;
namespace Ex5_03
{
    public class Person
    {
        public string Name { get; set; }
        public void Move()
        {   Console.WriteLine("Person is moving..."); }
        public void GoToPoint()
        {   Console.WriteLine($"{Name} go to point"); }
    }
    public class Animal
    {
        public string Name { get; set; }
        public void Move()
        {   Console.WriteLine("Animal is moving..."); }
        public void GoToPoint()
        {   Console.WriteLine($"{Name} go to point"); }
    }
```

Пример (продолжение)

```
class Program
    {
        static void Main(string[] args)
        {
            Person P = new Person { Name = "P" };
            P.Move();
            P.GoToPoint();

            Animal A = new Animal { Name = "A" };
            A.Move();
            A.GoToPoint();

            string End = Console.ReadLine();
        }
    }
}
```

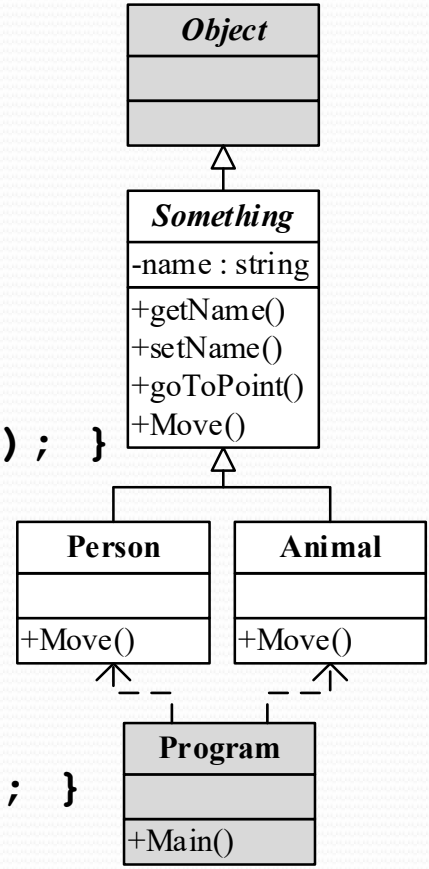


Пример реализации с абстрактным классом

Ex5.04

```
using System;

namespace Ex5_04
{
    abstract public class Something
    {
        public string Name { get; set; }
        public void GoToPoint()
        {  Console.WriteLine($"{Name} go to point"); }
        public abstract void Move();
    }
    public class Person:Something
    {
        override public void Move()
        { Console.WriteLine("Person is moving..."); }
    }
    public class Animal:Something
    {
        override public void Move()
        { Console.WriteLine("Animal is moving..."); }
    }
```

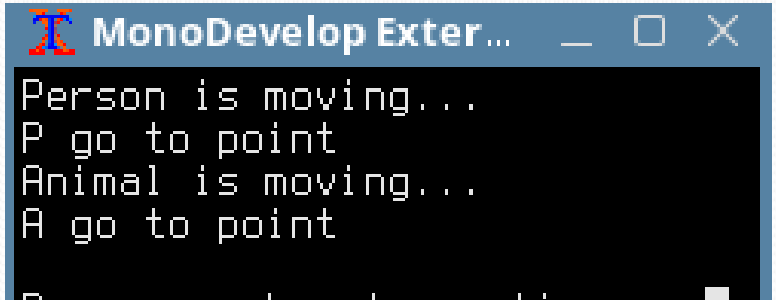


Пример (окончание)

```
class Program
    {
        static void Main(string[] args)
        {
            Person P = new Person {Name = "P"};
            P.Move();
            P.GoToPoint();

            Animal A = new Animal {Name = "A"};
            A.Move();
            A.GoToPoint();

            string End = Console.ReadLine();
        }
    }
}
```



5.4 Обобщения

Обобщение — средство языка C#, позволяющее создавать программный код, содержащий единственное (типизированное) решение задачи для различных типов, с его последующим применением для любого конкретного типа (int, float, char и т.д.). Аналог шаблонов C++, но подстановка конкретного типа в C# будет выполняться на этапе JIT–компиляции.

Обобщения могут быть применены к:

классам, структурам, интерфейсам, методам, делегатам.

Синтаксис описания обобщения класса:

```
class Имя_обобщения<Тип-параметр{ ,Тип-параметр}>
{
        // Реализация полей и методов класса с параметрами
}
```

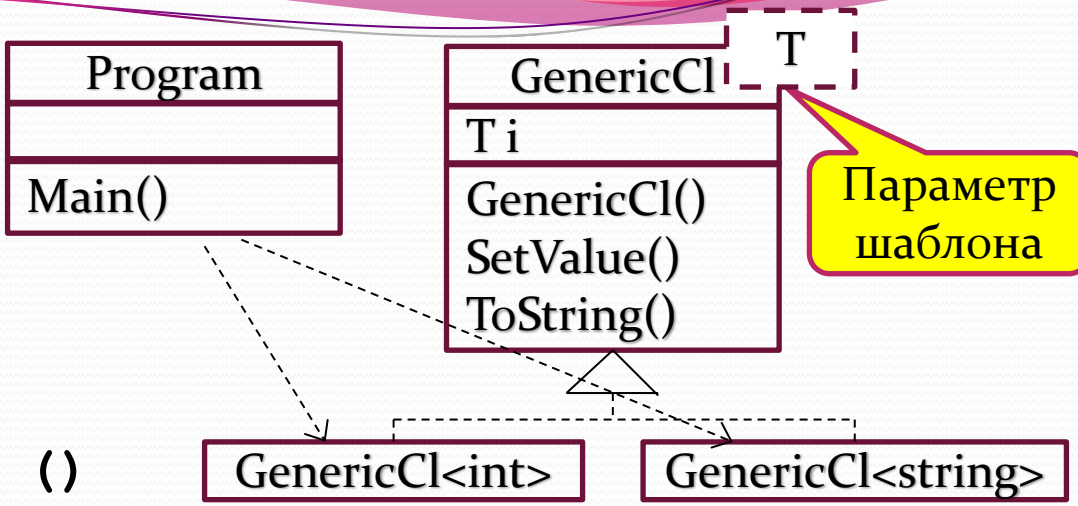
Синтаксис создания класса из обобщения:

```
Имя_обобщения <Тип {,Тип}> Имя =
        new Имя_обобщения <Тип {,Тип}>(Аргументы);
```

Пример Обобщения

Ex5.05

```
using System;
namespace Ex5_05
{
    class GenericCl<T>
    {
        private T i;
        public GenericCl ()
        {
            this.i = default(T);
        }
        public void SetValue(T param)
        {
            this.i = param;
        }
        public override string ToString()
        {
            return i.ToString();
        }
    }
```

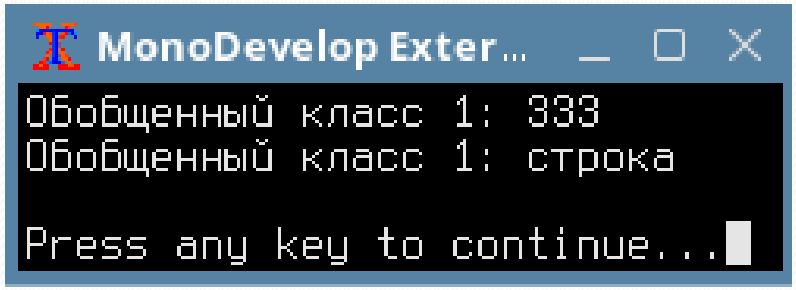


Задание значения типа по умолчанию позволяет проверить, было ли инициализировано поле

Пример (окончание)

```
class Program
{
   static void Main()
      {
         GenericCl<int> g1 = new GenericCl<int>();
         g1.SetValue(333);
         Console.WriteLine("Обобщенный класс 1: " + g1);

         GenericCl<string> g1Str =
                                 new GenericCl<string>();
         g1Str.SetValue("строка");
         Console.WriteLine("Обобщенный класс 1: "+g1Str);
   }
}
```



Ограничения подставляемых типов

Ограничение	Описание
`where T: struct`	Тип T должен быть типом-значением
`where T: class`	Тип T должен быть ссылочным типом
`where T: Class1`	Тип T должен наследоваться от класса Class1
`where T: Interface1`	Тип T должен реализовывать интерфейс Interface1
`where T1: T2`	При подстановке реальных типов в обобщенные типы T1 и T2, тип подставленный в T1 должен наследоваться от типа, подставленного в T2
`where T: new()`	Тип T должен иметь конструктор без параметров

Пример обобщенного метода с ограничением

```
static void GenericMethod1<T>(T param) where T:I1
{
    Console.WriteLine(param.I1_method());
}

GenericMethod1<I1_class>
                (new I1_class("Вызов обобщенного метода"));
```